

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 4
по дисциплине «Анализ алгоритмов»

Тема Программирование параллельных потоков

Студент Атаев К.А,

Группа ИУ7-52Б

Преподаватели Волкова Л.Л Строганов Ю. В.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Графы и связность 5

1.2 Удаление вершины 5

1.3 Последовательный алгоритм проверки связности 5

1.4 Параллельный алгоритм проверки связности 6

1.5 Основные понятия параллельного программирования 6

2 Конструкторская часть **8**

2.1 Алгоритм BFS 8

3 Технологическая часть **11**

3.1 Средства реализации 11

3.2 Схемы алгоритмов 11

3.3 Демонстрация работы программы 11

3.4 Функциональные тесты 12

4 Исследовательская часть **14**

4.1 Технические характеристики 14

4.2 Результаты исследования 14

ЗАКЛЮЧЕНИЕ **16**

ПРИЛОЖЕНИЕ А **18**

ВВЕДЕНИЕ

Цель лабораторной работы — сравнить последовательную и параллельную версии алгоритма проверки связности графа после удаления заданной на вход k-ой вершины.

Для достижения поставленной цели необходимо решить следующие задачи:

1) описать последовательный алгоритм проверки связности графа после удаления вершины k;

2) разработать параллельную версию алгоритма;

3) реализовать оба алгоритма;

4) выполнить сравнительный анализ зависимостей времени решения задачи проверки связности графа после удаления вершины k от размерности входных данных для реализации последовательной версии алгоритма и для реализации параллельной версии алгоритма, запущенного с единственным рабочим потоком;

5) выполнить сравнительный анализ зависимости времени работы параллельной версии алгоритма проверки связности графа после удаления вершины, от размерности входных данных при k вспомогательных потоков, k принимает значения: $1, 2, 4, 8, 16, 32, 64$;

1 Аналитическая часть

1.1 Графы и связность

Граф G определяется множеством вершин (1.1) и множеством рёбер (1.2):

$$V = \{v_1, v_2, \ldots, v_n\}, \tag{1.1}$$

$$E = \{(v_i, v_j) \mid v_i, v_j \in V\}, \tag{1.2}$$

Таким образом, граф задаётся парой множеств (1.3):

$$G = (V, E). \tag{1.3}$$

Путь в графе представляет собой последовательность вершин, каждые две соседние из которых соединены ребром. Граф называется связным, если для любых двух различных вершин существует путь, соединяющий их. Если хотя бы до одной вершины нельзя добраться из другой, граф считается несвязным [3].

1.2 Удаление вершины

В соответствии с вариантом задания требуется определить, сохраняет ли граф связность после удаления вершины k. Удаление вершины означает исключение её из множества V и удаление всех рёбер, в которых она участвовала. Полученный граф обозначается следующим образом (1.4):

$$G' = (V \setminus \{k\}, E'). \tag{1.4}$$

Граф G' считается связным, если оставшиеся вершины продолжают образовывать единую компоненту связности.

1.3 Последовательный алгоритм проверки связности

Для определения связности используется обход графа в ширину (Breadth-First Search, BFS). Этот метод позволяет установить, какие вершины достижимы из выбранной стартовой вершины.

Последовательный алгоритм включает следующие этапы:

1) считывается исходный граф G.
2) выполняется удаление вершины k и всех рёбер, связанных с ней, формируя граф G'.
3) выбирается стартовая вершина $s \in V \setminus \{k\}$, имеющая хотя бы одно ребро.
4) из вершины s выполняется обход BFS:
   — создаётся очередь вершин;

— каждая посещённая вершина отмечается в массиве признаков;

— в очередь помещаются все её соседи.

5) после завершения обхода проверяется, посещены ли все вершины графа G'.

Если посещены все вершины, граф считается связным.

1.4 Параллельный алгоритм проверки связности

Для ускорения вычислений используется параллельная модификация обхода BFS. Основная идея заключается в том, что несколько потоков могут одновременно извлекать вершины из общей очереди и обрабатывать их соседей.

Параллельный алгоритм включает следующие шаги:

1) определяется стартовая вершина s после удаления вершины k.

2) создаётся общая очередь, содержащая стартовую вершину.

3) запускается заданное число рабочих потоков.

4) каждый поток:

— извлекает вершину из очереди;

— обрабатывает всех её соседей;

— помещает непосещённых соседей в очередь.

5) работа потоков завершается, когда очередь пуста и все потоки находятся в состоянии ожидания.

Такой подход позволяет обрабатывать различные части графа параллельно, уменьшая время выполнения на больших входных данных.

1.5 Основные понятия параллельного программирования

При реализации параллельной версии алгоритма используются базовые понятия.

Процесс представляет собой исполняемую программу, обладающую собственным адресным пространством и ресурсами. Процессы изолированы друг от друга, что обеспечивает их независимость [2].

Поток является единицей выполнения внутри процесса. Потоки разделяют память процесса и могут выполняться параллельно, обрабатывая одни и те же данные [1, 2].

Мьютекс служит для обеспечения взаимного исключения. Он позволяет организовать доступ к участку программы таким образом, чтобы в определённый момент времени этот участок мог выполняться только одним потоком. Использование мьютекса предотвращает гонки данных при работе с общей памятью [1].

Синхронизация потоков представляет собой набор методов согласования действий потоков, необходимых при совместном доступе к данным. Она обеспечивает согласованность состояния программы и предотвращает нарушения целостности данных [2].

Вывод

В аналитической части были рассмотрены основные определения графов и связности, а также сформулированы последовательный и параллельный алгоритмы проверки связности графа после удаления вершины. Последовательная версия основана на обходе графа в ширину. Параллельная версия распределяет обработку между несколькими потоками и требует использования механизмов синхронизации для обеспечения корректного доступа к общим данным.

2 Конструкторская часть

2.1 Алгоритм BFS

Для проверки связности графа используется обход в ширину (BFS). Схема последовательной и параллельной реализаций представлена на рисунках 2.1 и 2.2.

Оценка вычислительной сложности BFS:

$$T_{\mathrm{BFS}} = O(|V| + |E|)\,, \tag{2.1}$$

где $|V|$ — число вершин, $|E|$ — число рёбер.

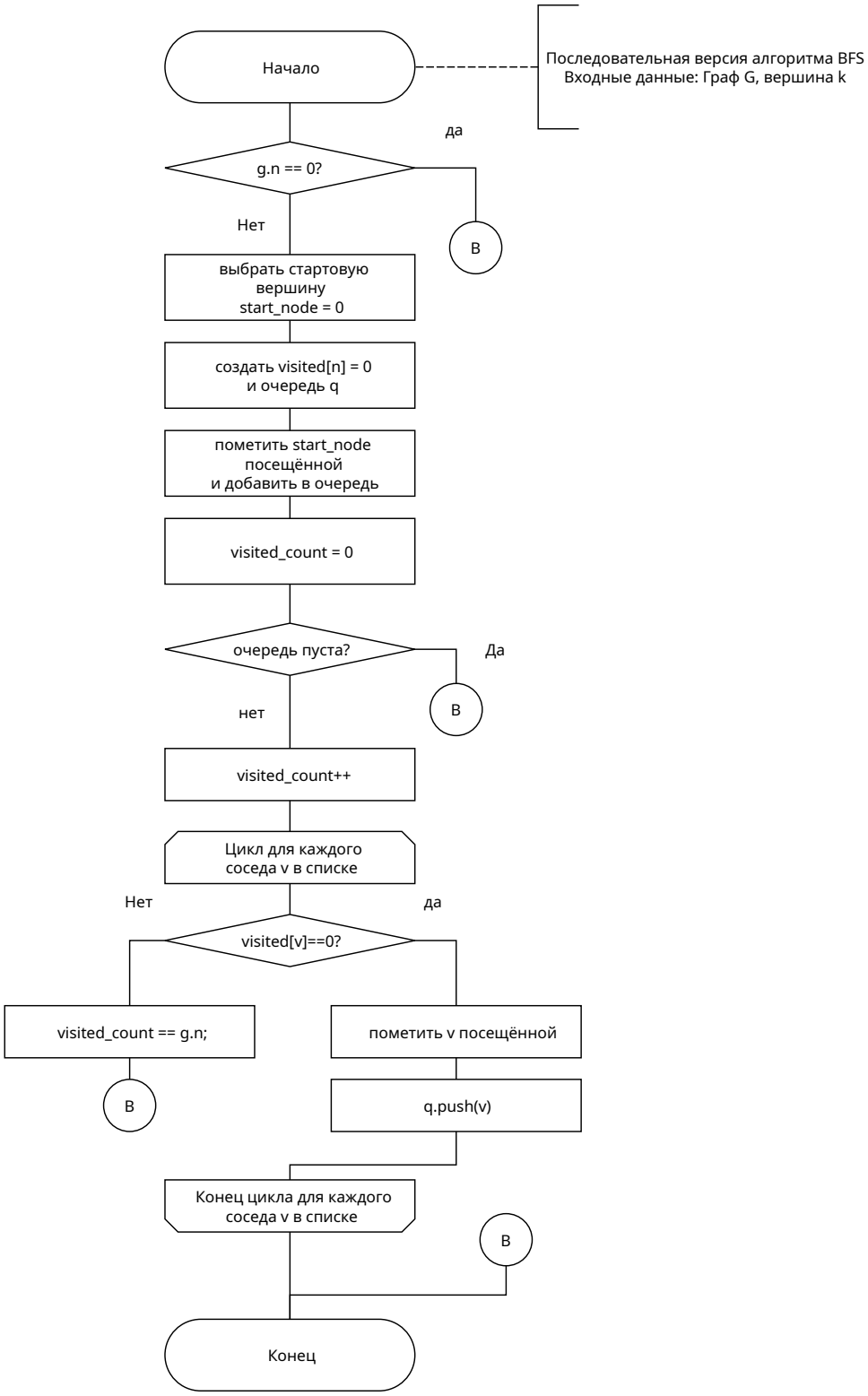


Рисунок 2.1 — Последовательная версия алгоритма BFS (проверка связности)

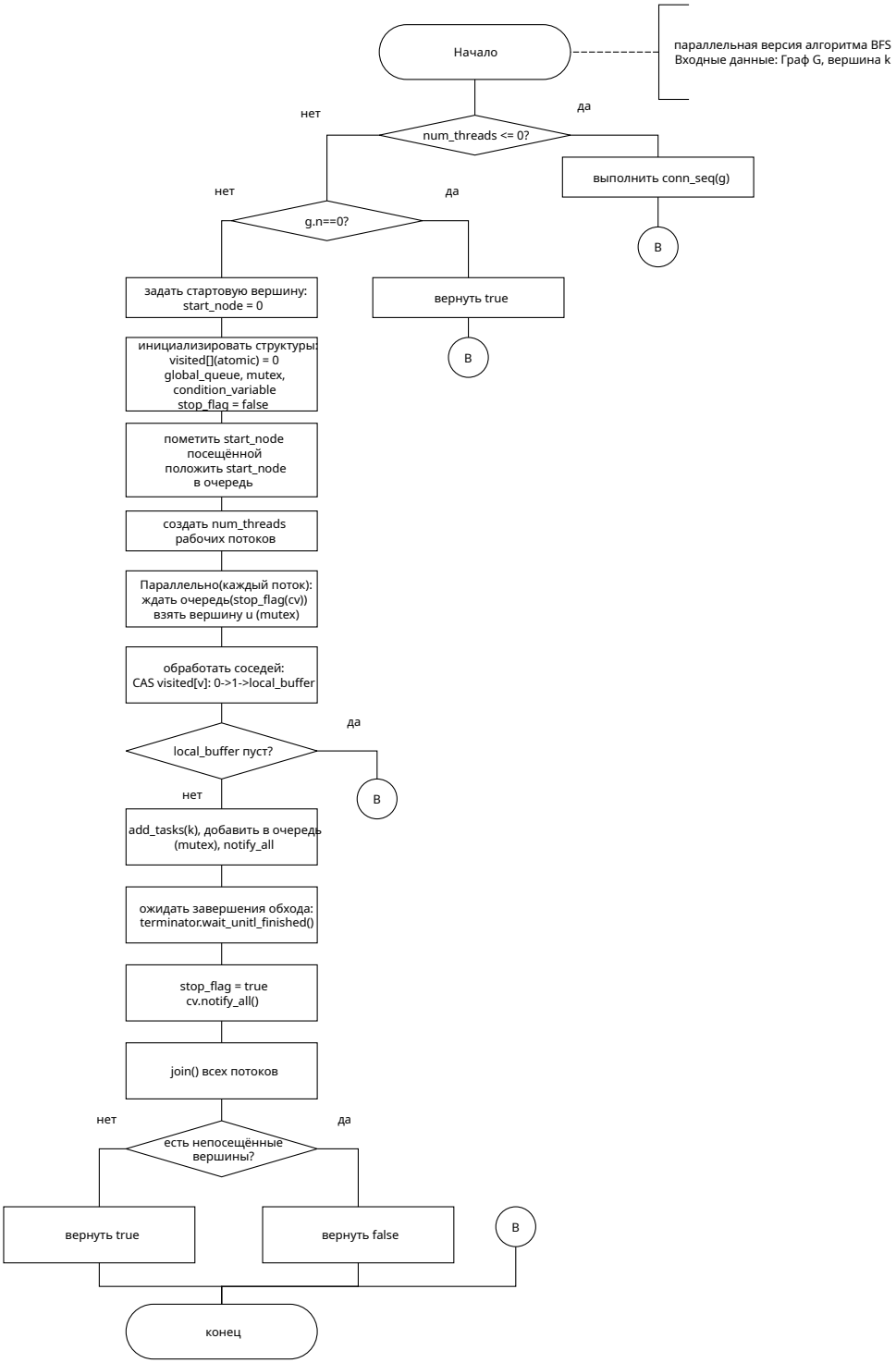


Рисунок 2.2 — Параллельная версия алгоритма BFS (проверка связности)

Вывод

В данной части были разработаны последовательный и параллельный алгоритмы обхода графа в ширину (BFS), предназначенные для проверки связности графа.

3 Технологическая часть

В данной части приведены средства реализации, реализации алгоритмов и функциональные тесты.

3.1 Средства реализации

Для реализации последовательного и параллельного алгоритмов проверки связности графа использовался язык программирования C++ (стандарт C++17) [4]. Параллельная версия реализована средствами стандартной библиотеки: `std::thread` для запуска потоков, `std::mutex` и $std :: condition_variable$ для синхронизации доступа к общей очереди, а также `std::atomic` для атомарной отметки посещённых вершин [4].

Сборка выполнялась компилятором `g++` из состава gcc [5].

Для получения временных замеров в исследовании использована функция `clock()` из заголовочного файла `<ctime>` [4].

3.2 Схемы алгоритмов

Схемы последовательной и параллельной реализаций проверки связности представлены на рисунках 2.1 и 2.2.

Оценка вычислительной сложности обхода в ширину (BFS):

$$T_{\mathrm{BFS}} = O(|V| + |E|)\,, \tag{3.1}$$

где $|V|$ — число вершин, $|E|$ — число рёбер.

3.3 Демонстрация работы программы

На рисунке 3.1 представлена демонстрация работы программы, выполняющей проверку связности графа параллельным алгоритмом BFS.

```
----------МЕНЮ----------
1. Последовательный алгоритм
2. Параллельный алгоритм
3. Получить замеры и сохранить CSV
4. Сгенерировать случайный граф (TXT)
5. Выход
Выберите пункт: 1
Файл (например graph.txt): c
Удаляемая вершина k (0..2999): 2
Не связный

----------МЕНЮ----------
1. Последовательный алгоритм
2. Параллельный алгоритм
3. Получить замеры и сохранить CSV
4. Сгенерировать случайный граф (TXT)
5. Выход
Выберите пункт: 2
Файл (например graph.txt): c
Удаляемая вершина k (0..2999): 1
Потоки (1,2,4,8,16,32,64): 16
Не связный
```

Рисунок 3.1 — Параллельная версия алгоритма BFS (проверка связности)

3.4 Функциональные тесты

Для проверки корректности работы последовательного и параллельного алгоритмов проверки связности графа были разработаны функциональные тесты. Тестирование выполнялось для различных классов графов: пустой граф, граф из одной вершины, связный граф, несвязный граф, а также графы, содержащие изолированные вершины. Результаты тестирования представлены в таблице 3.1.

В таблице граф задаётся списком смежности в виде $u \to \{v_1, v_2, \dots\}$, где u — номер вершины, а v_i — номера вершин, в которые существуют рёбра из u. Ожидаемый результат и получаемый результат представляются как булево значение: `true` — граф связен (все вершины достижимы из стартовой вершины 0), `false` — граф несвязен.

Таблица 3.1 — Пример представления результатов функциональных тестов алгоритмов проверки связности

Граф	**Ожидаемый результат**	**Последовательный алгоритм**	**Параллельный алгоритм**
Тест 1: $n = 0$ (пустой граф)	**Ожидаем:** `true` Пустой граф считается связным.	`conn_seq`**:** `true` Условие $g.n == 0$.	`conn_par`**:** `true` Условие $g.n == 0$.
Тест 2: $n = 4$ $0 \to 1$ $1 \to 0, 2$ $2 \to 1, 3$ $3 \to 2$	**Ожидаем:** `true` Все вершины достижимы из 0.	`conn_seq`**:** `true` Посещены: $0, 1, 2, 3$.	`conn_par`**:** `true` Посещены: $0, 1, 2, 3$.
Тест 3: $n = 5$ $0 \to 1$ $1 \to 0, 2$ $2 \to 1$ $3 \to 4$ $4 \to 3$	**Ожидаем:** `false` Вершины 3 и 4 не достижимы из 0.	`conn_seq`**:** `false` Посещены: $0, 1, 2$. не посетили: $3, 4$.	`conn_par`**:** `false` Посещены: $0, 1, 2$. не посетили: $3, 4$.

По результатам функционального тестирования установлено, что последовательный и параллельный алгоритмы корректно определяют связность графа на рассмотренных наборах входных данных: результаты, полученные при выполнении `conn_seq` и `conn_par`, совпадают с ожидаемыми значениями.

Вывод

В технологической части выбраны средства разработки и описана реализация последовательного и параллельного алгоритмов проверки связности графа. Также приведены схемы алгоритмов и указаны используемые средства измерения времени выполнения.

4 Исследовательская часть

В данной части приведены примеры работы программа, а также проведён сравнительный анализ алгоритмов при различных ситуациях на основе полученных данных.

4.1 Технические характеристики

Технические характеристики устройства, на котором выполнялось тестирование:

— операционная система Windows 11 домашняя [7];

— память 16 Гбайт;

— процессор Intel(R) Core(TM) i5-10300H CPU с тактовой частотой 2.50ГГц. [6]

4.2 Результаты исследования

Для сравнения эффективности последовательного и параллельного алгоритмов проверки связности графа были проведены замеры времени выполнения на графах различного размера. В качестве входных данных использовались графы с количеством вершин от 300 до 3000. Для каждого размера графа проводилось 100 измерений, после чего в качестве итогового значения времени использовалось среднее арифметическое.

Результаты представлены на рисунке 4.1.

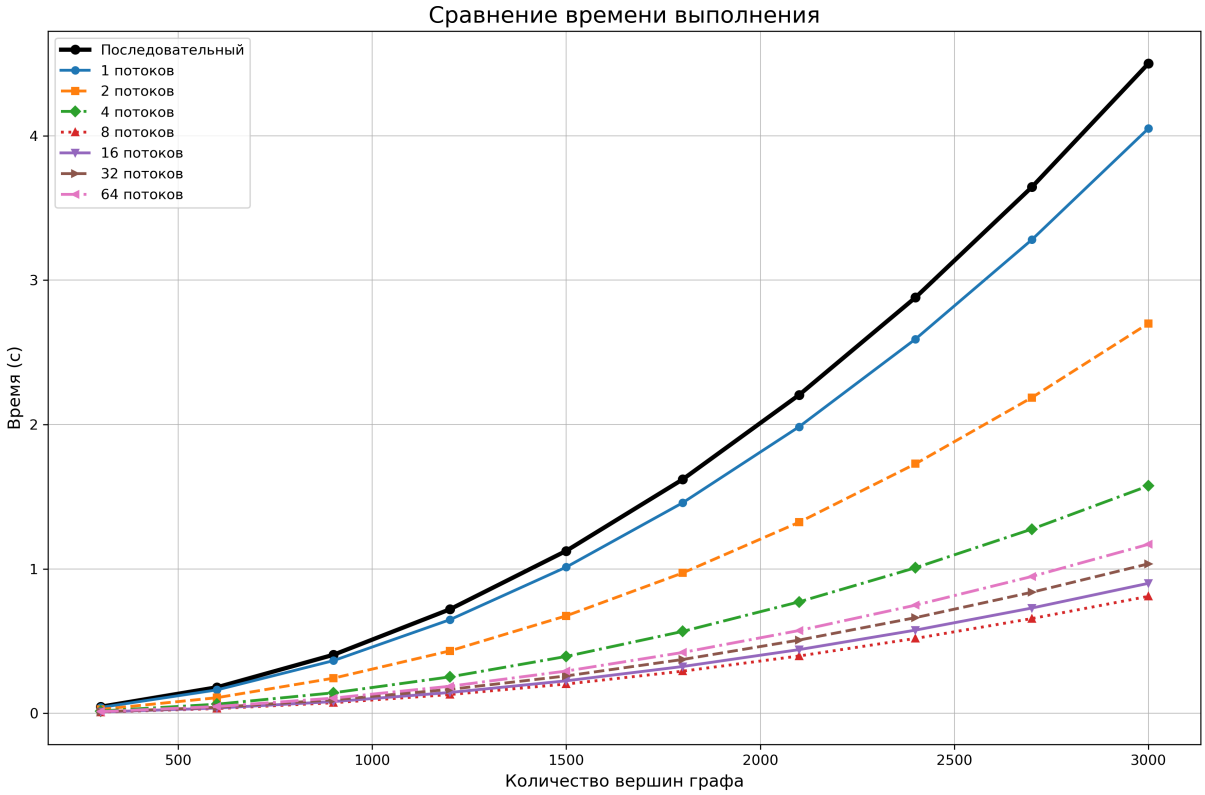


Рисунок 4.1 — Сравнение времени выполнения последовательного и параллельного алгоритмов проверки связности

Вывод

В результате проведённого исследования было установлено, что параллельная версия алгоритма проверки связности графа показывает лучшую производительность по сравнению с последовательной версией, особенно при увеличении размера входного графа. Однако при использовании большого количества потоков наблюдается уменьшение прироста производительности из-за накладных расходов на синхронизацию и конкуренцию за общие структуры данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была достигнута поставленная цель, заключающаяся в сравнении последовательной и параллельной версий алгоритма проверки связности графа после удаления заданной на вход k-ой вершины.

В рамках работы были выполнены следующие задачи:

1) описан последовательный алгоритм проверки связности графа после удаления вершины k;

2) разработана параллельная версия алгоритма проверки связности графа;

3) реализованы последовательная и параллельная версии алгоритма на языке программирования C++;

4) проведён сравнительный анализ зависимости времени выполнения задачи проверки связности графа после удаления вершины k от размерности входных данных для последовательной версии алгоритма и для параллельной версии, запущенной с одним рабочим потоком;

5) выполнен сравнительный анализ зависимости времени работы параллельной версии алгоритма проверки связности графа после удаления вершины k от размерности входных данных при использовании различного числа вспомогательных потоков $(1, 2, 4, 8, 16, 32, 64)$.

По результатам проведённого исследования установлено, что параллельная реализация алгоритма обеспечивает преимущество по времени выполнения на графах большого размера, при этом наибольшее ускорение достигается при использовании ограниченного числа потоков, сопоставимого с количеством логических ядер вычислительной системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Столяров А.В. Многопоточное программирование: учебное пособие. МФТИ, 2020.

2. Захаров В.В. Основы параллельного программирования. МГУ, 2015.

3. Валединский В. Д., Корнев А. А. Структуры данных и алгоритмы.

4. Working Draft, Standard for Programming Language C++ [Электронный ресурс] // Open-std : [сайт]. — Режим доступа: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf (дата обращения: 5.11.2025)

5. GNU Compiler Collection (GCC) documentation [Электронный ресурс] // GCC : [сайт]. — Режим доступа: https://gcc.gnu.org/onlinedocs/ (дата обращения: 6.11.2025)

6. Intel® Core™ i5-10300H Processor [Электронный ресурс] // Intel : [сайт]. — Режим доступа: https://ark.intel.com/content/www/us/en/ark/products/201837/intel-core-i5-10300h-processor-8m-cache-up-to-4-50-ghz.html (дата обращения: 7.11.2025)

7. Windows 11 Home [Электронный ресурс] // Microsoft : [сайт]. — Режим доступа: https://www.microsoft.com/ru-ru/windows/windows-11 (дата обращения: 8.11.2025)

ПРИЛОЖЕНИЕ А

```
bool conn_seq(const Graph& g)
{
    if (g.n == 0) return true;

    int start_node = 0;

    std::vector<unsigned char> visited(g.n, 0);
    std::queue<int> q;

    visited[start_node] = 1;
    q.push(start_node);

    int visited_count = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        visited_count++;

        for (int v : g.adj[u]) {
            if (!visited[v]) {
                visited[v] = 1;
                q.push(v);
            }
        }
    }

    return visited_count == g.n;
}
```

Листинг А.1 — Реализация последовательного алгоритма проверки связности (BFS)

```
class TerminationDetector {
    std::atomic<int> active_tasks_{0};
    std::mutex mtx_;
    std::condition_variable cv_;

public:
    void add_tasks(int n) {
        active_tasks_.fetch_add(n, std::memory_order_relaxed);
    }

    void task_completed() {
        if (active_tasks_.fetch_sub(1, std::memory_order_acq_rel) == 1)
            {
            std::lock_guard<std::mutex> lock(mtx_);
            cv_.notify_all();
        }
    }

    void wait_until_finished() {
        std::unique_lock<std::mutex> lock(mtx_);
        cv_.wait(lock, [&] {
            return active_tasks_.load(std::memory_order_acquire) == 0;
        });
    }
};
```

Листинг А.2 — Детектор завершения задач

```
bool conn_par(const Graph& g, int num_threads)
{
    if (num_threads <= 0) return conn_seq(g);
    if (g.n == 0) return true;

    int start_node = 0;

    std::vector<std::atomic<unsigned char>> visited(g.n);
    for (int i = 0; i < g.n; ++i) {
        visited[i].store(0, std::memory_order_relaxed);
    }

    std::queue<int> global_queue;
    std::mutex queue_mutex;
    std::condition_variable cv_worker;
    std::atomic<bool> stop_flag{false};

    TerminationDetector terminator;

    visited[start_node].store(1, std::memory_order_relaxed);
    global_queue.push(start_node);
    terminator.add_tasks(1);

    auto worker_func = [&]() {
        std::vector<int> local_buffer;
        local_buffer.reserve(128);

        while (true) {
            int u = -1;

            {
                std::unique_lock<std::mutex> lock(queue_mutex);
                cv_worker.wait(lock, [&] {
                    return !global_queue.empty() ||
                           stop_flag.load(std::memory_order_relaxed);
                });

                if (global_queue.empty()) {
                    if (stop_flag.load(std::memory_order_relaxed)) {
                        return;
```

```
                }
                continue;
            }

            u = global_queue.front();
            global_queue.pop();
        }

        for (int v : g.adj[u]) {
            unsigned char expected = 0;
            if (visited[v].load(std::memory_order_relaxed) == 0 &&
                visited[v].compare_exchange_strong(
                    expected, 1, std::memory_order_relaxed)) {
                local_buffer.push_back(v);
            }
        }

        if (!local_buffer.empty()) {
            terminator.add_tasks((int)local_buffer.size());
            {
                std::lock_guard<std::mutex> lock(queue_mutex);
                for (int node : local_buffer) {
                    global_queue.push(node);
                }
            }
            cv_worker.notify_all();
            local_buffer.clear();
        }

        terminator.task_completed();
    }
};

std::vector<std::thread> threads;
threads.reserve(num_threads);
for (int i = 0; i < num_threads; ++i) {
    threads.emplace_back(worker_func);
}

terminator.wait_until_finished();
```

```
    stop_flag.store(true, std::memory_order_relaxed);
    cv_worker.notify_all();

    for (auto& t : threads) {
        if (t.joinable()) t.join();
    }

    for (int i = 0; i < g.n; ++i) {
        if (visited[i].load(std::memory_order_relaxed) == 0) {
            return false;
        }
    }

    return true;
}
```

Листинг А.3 — Реализация параллельного алгоритма проверки связности (BFS)

```
#include <ctime>

static inline double ticks_to_ms(clock_t ticks) {
    return 1000.0 * static_cast<double>(ticks) /
           static_cast<double>(CLOCKS_PER_SEC);
}

double measure_execution_time(bool (*f)(const Graph&), const Graph& g)
   {
    clock_t t0 = clock();

    volatile bool ok = f(g);
    (void)ok;

    clock_t t1 = clock();

    double ms = ticks_to_ms(t1 - t0);
    return (ms < 0.0) ? 0.0 : ms;
}

double measure_execution_time(bool (*f)(const Graph&, int),
                              const Graph& g, int threads) {
    clock_t t0 = clock();

    volatile bool ok = f(g, threads);
    (void)ok;

    clock_t t1 = clock();

    double ms = ticks_to_ms(t1 - t0);
    return (ms < 0.0) ? 0.0 : ms;
}
```

Листинг А.4 — Реализация замеров времени выполнения (clock)